HIREAI ENGINEERING BLUEPRINT • V1.0

AI Engine Architecture & Backend Integration Specification

Technical blueprint for integrating the standalone FastAPI AI Engine with the Spring Boot 3.5 & PostgreSQL backend.

6 Modules
AI CORE CAPABILITIES

8 Endpoints
REST API SURFACE

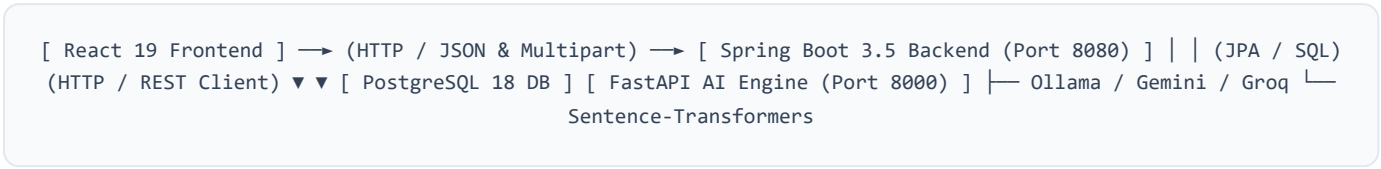
Spring Boot 3.5
TARGET ORCHESTRATOR

Target Systems: Spring Boot 3.5.5 (Port 8080) • FastAPI AI Engine (Port 8000)

Author: HireAI Core Team • August 2026

1. Executive Summary & System Topology

The HireAI platform architecture enforces a strict separation of concerns between **transactional business logic** (Spring Boot) and **stateless AI processing** (FastAPI microservice). The AI Engine runs independently, wrapping local LLMs (Ollama / Llama 3.1:8b), cloud providers (Gemini / Groq), and local embedding models.



Key Architectural Rules:

- **Database Isolation:** The AI Engine never connects directly to PostgreSQL. Spring Boot acts as the sole data gatekeeper.
- **Text Extraction Boundary:** The AI Engine does not accept raw PDF/DOCX files. Spring Boot extracts plain text and sends JSON.
- **Sync vs Async Models:** Resume parsing uses an asynchronous polling pattern (30-90s on local models), while other 5 modules are synchronous.

2. Complete Scan of the 6 AI Engine Modules

#	Module Name	Endpoint	Method	Pattern	Processing Engine
1	**JD Generator**	`/api/v1/jd/generate`	POST	SYNC	LLM (Llama 3.1:8b / Gemini / Groq)
2	**Resume Parser**	`/api/v1/resumes/parse` `/api/v1/resumes/status/{id}`	POST GET	ASYNC POLL	LLM Zero-Shot Extraction + In-Memory Thread Pool
3	**Match Engine**	`/api/v1/match/score`	POST	SYNC	Sentence Transformers (`all-MiniLM-L6-v2`)
4	**AI Chatbot**	`/api/v1/chatbot/message`	POST	SYNC	LLM Conversational State Machine
5	**Interview AI**	`/api/v1/interview/questions` `/api/v1/interview/evaluate`	POST POST	SYNC	LLM Technical Assessment & Grading
6	**Decision Engine**	`/api/v1/decision/finalize`	POST	SYNC	Deterministic Weighted Formula (No LLM)

Module 1: Job Description (JD) Generator

Generates structured role overview, responsibilities, required skills, and sample technical questions from title and experience level.

```
// POST /api/v1/jd/generate
// Request Body:
{
  "jobTitle": "Backend Engineer",
  "requiredSkills": ["Java", "Spring Boot", "PostgreSQL", "Docker"],
  "experienceLevel": "3-5 years"
}

// Response Body (200 OK):
{
  "description": "Lead the development of scalable microservices...",
  "responsibilities": ["Design and maintain high-throughput REST APIs", "Collaborate with frontend teams"],
  "mustHaveSkills": ["Java", "Spring Boot", "PostgreSQL", "Docker"],
  "niceToHaveSkills": ["Kubernetes", "Redis", "Kafka"],
  "interviewQuestions": ["Explain transaction propagation in Spring Boot.", "How do you handle connection pools in
}
```

Module 2: Resume Parser (Async Job Polling)

Accepts raw extracted text from a candidate's resume and performs zero-shot entity extraction in a background thread.

```
// 1. Start Parse Job: POST /api/v1/resumes/parse
{
  "candidateId": "cand_1029",
  "resumeText": "Rohan Mehta\nSenior Java Developer\nSkills: Java, Spring Boot, Postgres..."
}
// Returns immediately: { "jobId": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "status": "processing" }

// 2. Poll Status: GET /api/v1/resumes/status/3fa85f64-5717-4562-b3fc-2c963f66afa6
{
  "jobId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "status": "complete",
  "result": {
    "candidateId": "cand_1029",
    "skills": ["Java", "Spring Boot", "PostgreSQL", "Docker"],
    "yearsExperience": 4.0,
    "education": [{"degree": "B.Tech Computer Science", "institution": "Pune Institute of Tech", "year": "2020"}],
    "projects": [{"name": "Order Management System", "description": "Microservices with Spring Cloud"}],
    "domain": "fintech",
    "currentRole": "Senior Java Developer",
    "parseStatus": "success"
  }
}
```

3. Backend Audit & Gap Analysis

⚠️ **Critical Mismatches Identified in Existing Backend:**

- `LlmAtsClient.java` currently targets `/api/v1/ats/score` (configured in `AtsProperties.java`). **This endpoint does not exist on the AI engine.** It must be updated to `/api/v1/match/score`.
- `LlmAtsClient.java` sends full `CandidateInfo` and `JobInfo` nested objects. The AI Engine accepts `{ "resumeSkills": [...], "jobSkills": [...] }`.
- Resume file uploading exists in DTO (`ResumeUploadRequest`), but no controller or text extractor (Apache Tika) exists.
- No asynchronous polling mechanism exists in Spring Boot to poll for resume parse completion.

Summary of Required Modifications

Component	Current State	Target State
AtsProperties.java	`serviceUrl = ".../api/v1/ats/score"`	Point base URL to `http://localhost:8000` and use `/api/v1/match/score`
LlmAtsClient.java	Single purpose ATS client with wrong payload	Replace with comprehensive `AiEngineClient.java` supporting all 6 modules
Resume.java Entity	Only stores file metadata	Add fields for `rawText` , `aiJobId` , `parsedRole` , `parsedExperience` , `parsedDataJson`
JobApplication.java	Stores only ATS score	Add `interviewScore` , `chatbotScore` , `finalAiScore` , `aiClassification` , `aiExplanation`
New Entities	None	Create `ChatConversation` , `ChatMessage` , `InterviewAssessment` , `InterviewQuestionAnswer`

4. Backend Schema & Entity Modifications

1. Modification to `Resume.java`

```
@Getter @Setter @Entity @Table(name = "resumes")
public class Resume extends BaseEntity {
    // Existing fields: id, originalFileName, storedFileName, fileType, fileSize, filePath, resumeStatus...

    @Column(name = "raw_text", columnDefinition = "TEXT")
    private String rawText;

    @Column(name = "ai_job_id", length = 64)
    private String aiJobId;

    @Column(name = "parsed_domain", length = 100)
    private String parsedDomain;

    @Column(name = "parsed_role", length = 100)
    private String parsedRole;

    @Column(name = "parsed_experience", precision = 4, scale = 1)
    private BigDecimal parsedExperience;

    @Column(name = "parsed_data_json", columnDefinition = "TEXT")
    private String parsedDataJson;
}
```

2. Modification to `JobApplication.java`

```
@Getter @Setter @Entity @Table(name = "job_applications")
public class JobApplication extends BaseEntity {
    // Existing fields: id, job, candidate, status, atsMatchScore, matchingSkills, missingSkills...

    @Column(name = "interview_score")
    private Integer interviewScore;

    @Column(name = "chatbot_score")
    private Integer chatbotScore;

    @Column(name = "final_ai_score")
    private Integer finalAiScore;

    @Column(name = "ai_classification", length = 20)
    private String aiClassification; // SHORTLIST, HOLD, REJECT

    @Column(name = "ai_explanation", length = 2000)
    private String aiExplanation;
}
```

3. New Entity: `ChatConversation.java` & `ChatMessage.java`

```
@Entity @Table(name = "chat_conversations")
public class ChatConversation extends BaseEntity {
    @Id @GeneratedValue private UUID id;
    @OneToOne @JoinColumn(name = "candidate_id", nullable = false) private Candidate candidate;
    @Column(nullable = false) private boolean complete = false;
    @OneToMany(mappedBy = "conversation", cascade = CascadeType.ALL) private List<ChatMessage> messages;
}

@Entity @Table(name = "chat_messages")
public class ChatMessage extends BaseEntity {
    @Id @GeneratedValue private UUID id;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "conversation_id") private ChatConversation conversation;
    @Column(nullable = false, length = 20) private String role; // "bot" | "candidate"
    @Column(nullable = false, columnDefinition = "TEXT") private String text;
}
```

5. Unified Spring Boot AI Client

The centralized `AiEngineClient` encapsulates all communications with the Python AI Engine:

```
package com.vionsys.hireai.ai.client;

import java.time.Duration;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.MediaType;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;
import com.vionsys.hireai.ai.dto.match.*;
import com.vionsys.hireai.ai.dto.resume.*;
import com.vionsys.hireai.ai.dto.jd.*;
import com.vionsys.hireai.ai.dto.chatbot.*;
import com.vionsys.hireai.ai.dto.interview.*;
import com.vionsys.hireai.ai.dto.decision.*;

@Component
public class AiEngineClient {

    private final RestClient restClient;

    public AiEngineClient(
            @Value("${ai-engine.base-url:http://localhost:8000}") String baseUrl,
            @Value("${ai-engine.timeout-ms:30000}") int timeoutMs) {

        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(Duration.ofMillis(timeoutMs));
        factory.setReadTimeout(Duration.ofMillis(timeoutMs));

        this.restClient = RestClient.builder()
                .baseUrl(baseUrl)
                .requestFactory(factory)
                .build();
    }

    public AiJdGenerateResponse generateJd(AiJdGenerateRequest req) {
        return restClient.post().uri("/api/v1/jd/generate")
                .contentType(MediaType.APPLICATION_JSON).body(req)
                .retrieve().body(AiJdGenerateResponse.class);
    }

    public AiJobAcceptedResponse submitResumeForParsing(AiResumeParseRequest req) {
        return restClient.post().uri("/api/v1/resumes/parse")
                .contentType(MediaType.APPLICATION_JSON).body(req)
                .retrieve().body(AiJobAcceptedResponse.class);
    }

    public AiJobStatusResponse checkResumeParseStatus(String jobId) {
        return restClient.get().uri("/api/v1/resumes/status/{jobId}", jobId)
                .retrieve().body(AiJobStatusResponse.class);
    }

    public AiMatchScoreResponse calculateMatchScore(AiMatchScoreRequest req) {
        return restClient.post().uri("/api/v1/match/score")
                .contentType(MediaType.APPLICATION_JSON).body(req)
                .retrieve().body(AiMatchScoreResponse.class);
    }

    public AiChatMessageResponse sendChatMessage(AiChatMessageRequest req) {
        return restClient.post().uri("/api/v1/chatbot/message")
                .contentType(MediaType.APPLICATION_JSON).body(req)
```

```
                .retrieve().body(AiChatMessageResponse.class);
    }

    public AiInterviewQuestionsResponse generateQuestions(AiInterviewQuestionsRequest req) {
        return restClient.post().uri("/api/v1/interview/questions")
                .contentType(MediaType.APPLICATION_JSON).body(req)
                .retrieve().body(AiInterviewQuestionsResponse.class);
    }

    public AiInterviewEvaluateResponse evaluateAnswers(AiInterviewEvaluateRequest req) {
        return restClient.post().uri("/api/v1/interview/evaluate")
                .contentType(MediaType.APPLICATION_JSON).body(req)
                .retrieve().body(AiInterviewEvaluateResponse.class);
    }

    public AiDecisionResponse finalizeDecision(AiDecisionRequest req) {
        return restClient.post().uri("/api/v1/decision/finalize")
                .contentType(MediaType.APPLICATION_JSON).body(req)
                .retrieve().body(AiDecisionResponse.class);
    }
}
```

6. Phased Implementation Roadmap

Phase	Focus Area	Key Deliverables
Phase 1	**Core Setup & Match Engine Fix**	• Fix `AtsProperties.java` to point to `http://localhost:8000/api/v1/match/score` • Implement `AiEngineClient.java` and base configuration • Align `AtsMatchScoringService` with `AiMatchScoreRequest` / `Response`
Phase 2	**Resume Text Extraction & Async Parser**	• Add `org.apache.tika` to `pom.xml` • Build `DocumentTextExtractionService` for PDF/DOCX parsing • Implement `POST /api/v1/candidate/resume/upload` and background status polling worker
Phase 3	**AI Job Description Generator**	• Build `AiJdGenerateRequest` and `AiJdGenerateResponse` DTOs • Expose `POST /api/v1/recruiter/ai/jd/generate` in `JobController` • Connect with Recruiter Frontend Job Creation form
Phase 4	**Screening Chatbot State Machine**	• Create `ChatConversation` and `ChatMessage` entities & repositories • Expose `POST /api/v1/candidate/chat/message` & `GET /history` • Auto-update candidate CTC, notice period, and availability on completion
Phase 5	**Interview AI & Decision Engine**	• Create `InterviewAssessment` entities & repositories • Implement question generation and automated answer scoring endpoints • Connect `/api/v1/decision/finalize` to composite hiring pipeline

✅ **Status:** Technical specification complete. Ready for phased implementation upon confirmation.